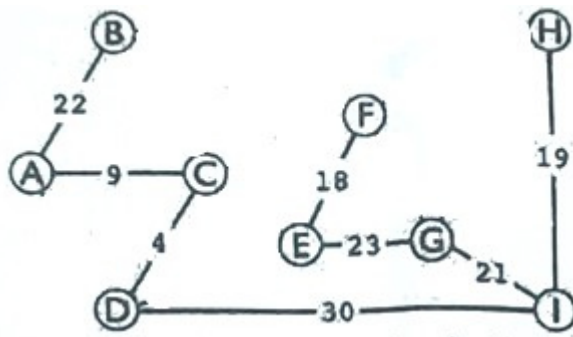


DESIGN AND ANALYSIS OF ALGORITHMUM

(Spring 2024 & FALL 2023)

Solution by (CH MUZAMMIL)

A) Show step-by-step execution of Kruskal's algorithm for the graph given below where start vertex is B.



Answer:

SOLUTION with Kruskal's Algorithm is not possible because Kruskal's algorithm no starts from specific vertex.

Here, is CHATGPT GENERATED SOLUTION

1) Initialize MST and Add Edges:

- Start with B.
- Add (A,B) : 22 (Start point is B)
- Add (C,D) : 4
- Add (A,C) : 9
- Add (E,F) : 18
- Add (I,H) : 19
- Add (G,I) : 21
- Add (E,G) : 23
- Add (D,I) : 30

Final MST:

Edges in the MST:

- (A,B) : 22 (Start point is B)
- (C,D) : 4
- (A,C) : 9
- (E,F) : 18
- (I,H) : 19
- (G,I) : 21

- (E,G) : 23
- (D,I) : 30

B) Solve the following recurrences using master theorem.

a) $T(n) = 2T(n/3) + n$

b) $T(n) = 2T(n/2) + n^2$

Answer:

1. $T(n) = 2T(n/3) + n$

Compare with $T(n) = aT(n/b) + f(n)$:

- $a = 2, b = 3, f(n) = n$, and $p = \log_b a = \log_3 2 \approx 0.63$.
- $f(n) = n^1$, so $p < 1$.
- $f(n)$ dominates, so $T(n) = \Theta(n)$.

2. $T(n) = 2T(n/2) + n^2$

Compare with $T(n) = aT(n/b) + f(n)$:

- $a = 2, b = 2, f(n) = n^2$, and $p = \log_b a = \log_2 2 = 1$.
- $f(n) = n^2$, so $p < 2$.
- $f(n)$ dominates, so $T(n) = \Theta(n^2)$.

C) What is coin change problem? Which technique is used to solve this problem?

Answer:

The **coin change problem** involves finding ways to make a target amount using a set of coin denominations. It has two main variations:

1. **Counting the number of ways** to form the amount.
2. **Finding the minimum number of coins** needed to form the amount.

Technique Used:

Dynamic Programming (DP) is used to solve the problem.

- For counting ways, a DP table is created where each entry represents the number of ways to make a specific amount.
 - For minimizing coins, a DP table stores the minimum number of coins required for each amount, updating using the recurrence relation $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$.
-

D) Explain Max-Heap. What is cost of deletion of an element within max-Heap?

Answer:

A **Max-Heap** is a binary tree where each parent node's value is greater than or equal to its children's values. The largest element is always at the root, and the tree is a complete binary tree (filled from left to right).

Cost of Deletion:

To delete the root (max element), the following steps are performed:

1. Replace the root with the last element.
2. Remove the last element.
3. Heapify down (adjust the tree) to restore the heap property.

The time complexity for deletion is $O(\log n)$, where n is the number of elements in the heap, because we may need to traverse the height of the tree.

E) What is the role of pivot in Quick sort? How selection of pivot effects running time of Quick Sort? What is upper-bound for running time of Quick sort?

Answer:

Role of pivot: The pivot partitions the array into two subarrays such that all elements smaller than the pivot are on the left and all greater elements are on the right.

Effect of pivot selection:

- If the pivot is well-chosen (near the median), Quick Sort has optimal performance.
- If poorly chosen (e.g., always the smallest or largest element), it leads to unbalanced partitions, increasing running time.

Upper bound: $O(n^2)$ (worst case), though average case is $O(n \log n)$.

F) Consider a hash-table of 15 elements. Place the following values in a Hash-Table if the hash-function is $X \bmod 5$. Use linear-probing to resolve the collision.

12	10	0	5	9	12	25	41	10	3	5	7	9
----	----	---	---	---	----	----	----	----	---	---	---	---

Answer:

Hash Function:

$X \bmod 5$ determines the index.

Linear Probing resolves collisions by placing the value in the next available slot.

Values and Placement:

Element	Hash ($X \bmod 5$)	Placement Process
12 10 0	$12 \bmod 5 = 2$	Place in index 2
5 9 12	$10 \bmod 5 = 0$	Place in index 0
25 41	$0 \bmod 5 = 0$	Collision, place in next available slot (index 1)
10 3 5 7	$5 \bmod 5 = 0$	Collision, move to index 3
9	$9 \bmod 5 = 4$	Place in index 4
	$12 \bmod 5 = 2$	Collision, move to index 5
	$25 \bmod 5 = 0$	Collision, move to next available slot (index 6)
	$41 \bmod 5 = 1$	Collision, move to next available slot (index 7)
	$10 \bmod 5 = 0$	Collision, move to next available slot (index 8)
	$3 \bmod 5 = 3$	Collision, move to next available slot (index 9)
	$5 \bmod 5 = 0$	Collision, move to next available slot (index 10)
	$7 \bmod 5 = 2$	Collision, move to next available slot (index 11)
	$9 \bmod 5 = 4$	Collision, move to next available slot (index 12)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Value	10	0	12	5	9	12	25	41	10	3	5	7	9	—	—

LONG QUESTIONS

A) What is the recurrence relation of algorithm to find Fibonacci series? Solve the recurrence and find upper and lower bounds.

Answer:

Recurrence Relation for Fibonacci:

The Fibonacci sequence is defined as:

$F(n) = F(n-1) + F(n-2)$, with $F(0) = 0$ and $F(1) = 1$.

This means:

- To compute $F(n)$, you need the sum of the previous two Fibonacci numbers ($F(n-1)$ and $F(n-2)$).

Solving the Recurrence Relation:

The solution to this recurrence relation is derived mathematically, but the **time complexity** depends on the approach used to calculate Fibonacci:

1. **Recursive Algorithm:**

- Recursive calls for $F(n-1)$ and $F(n-2)$ lead to overlapping subproblems.
- The recurrence has an **exponential growth** due to repeated work, making it:
Time Complexity: $T(n) = O(2^n)$.

2. **Dynamic Programming (Iterative or Memoized):**

- Solves the recurrence iteratively or by caching intermediate results.
- Each Fibonacci value is calculated only once.

Upper and Lower Bounds:

- **Upper Bound:** The recursive algorithm has a time complexity of $O(2^n)$, which is the worst-case.
- **Lower Bound:** The dynamic programming approach reduces the complexity to $O(n)$, which is the best-case.

Conclusion:

- Recurrence Relation: $F(n) = F(n-1) + F(n-2)$.

- Recursive approach: **Exponential time $O(2^n)$** .
- Optimized approach: **Linear time $O(n)$** using dynamic programming.

B) For what types of problems, topological sort algorithm is used as a solution? Briefly explain the algorithm with an example.

Answer:

Topological Sort Used For:

Topological Sort is used for problems involving **dependencies** or **ordering tasks**. Specifically, it applies to:

1. **Directed Acyclic Graphs (DAGs):**
 - The graph must be directed and must not have cycles.
2. **Common Applications:**
 - Task scheduling (e.g., building a project where tasks depend on each other).
 - Determining the order of courses in a curriculum (e.g., prerequisites).
 - Resolving symbol dependencies in compilers.

How the Algorithm Works:

1. **Input:** A Directed Acyclic Graph (DAG).
2. **Output:** A linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u appears before v .

Algorithm Steps:

Find Indegree: Calculate the indegree (number of incoming edges) of each vertex.

Start with Zero Indegree: Add vertices with indegree 0 to a queue (or stack).

Process Vertices:

- Remove a vertex from the queue and add it to the result.
- Reduce the indegree of its neighbors by 1.
- If any neighbor's indegree becomes 0, add it to the queue.

Repeat until all vertices are processed.

Check for Cycles: If vertices remain unprocessed, the graph is not a DAG (contains cycles).

Example:

Graph:

Tasks: A, B, C, D, E, F

Edges: $A \rightarrow D$, $B \rightarrow D$, $C \rightarrow E$, $D \rightarrow E$, $E \rightarrow F$

Steps:

1. **Indegree Table:**

A:0, B:0, C:0, D:2, E:2, F:1

2. **Start with Zero Indegree:**

Add A, B, C to the queue.

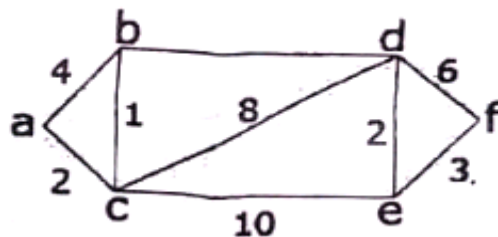
3. **Process A:**
 - Add A to the result: [A].
 - Reduce D's indegree to 1.
4. **Process B:**
 - Add B to the result: [A, B].
 - Reduce D's indegree to 0 → Add D to the queue.
5. **Process C:**
 - Add C to the result: [A, B, C].
 - Reduce E's indegree to 1.
6. **Process D:**
 - Add D to the result: [A, B, C, D].
 - Reduce E's indegree to 0 → Add E to the queue.
7. **Process E:**
 - Add E to the result: [A, B, C, D, E].
 - Reduce F's indegree to 0 → Add F to the queue.
8. **Process F:**
 - Add F to the result: [A, B, C, D, E, F].

Final Output:

A valid linear order of tasks. Example: [A, B, C, D, E, F].

This ensures dependencies are resolved before proceeding.

C) Execute dijkstra's algorithm for the following graph and write all the shortest paths and their costs for given source and destination in the table below.



Source	Destination	Path	Cost
--------	-------------	------	------

A	F		
A	G		
A	H		
A	C		
A	D		

Answer:

AS the cost of edge between b and d vertex is not given. So that edge is ignored.

Explored	Unexplored
a^0	$a^0 \ b^{\infty} \ c^{\infty} \ d^{\infty} \ e^{\infty} \ f^{\infty}$
$a^0 \ c^{2a}$	$b^{4a} \ c^{2a} \ d^{\infty} \ e^{\infty} \ f^{\infty}$
$a^0 \ c^{2a} \ b^{3c}$	$b^{3c} \ d^{10c} \ e^{12c} \ f^{\infty}$
$a^0 \ c^{2a} \ b^{3c} \ d^{10c}$	$d^{10c} \ e^{12c} \ f^{\infty}$
$a^0 \ c^{2a} \ b^{3c} \ d^{10c} \ e^{12c}$	$e^{12c} \ f^{15e}$
$a^0 \ c^{2a} \ b^{3c} \ d^{10c} \ e^{12c} \ f^{15e}$	f^{15e}

From A: B : $A \rightarrow B$

C: $A \rightarrow C$ D: $A \rightarrow C$

$\rightarrow D$ E: $A \rightarrow C \rightarrow E$

F: $A \rightarrow C \rightarrow E \rightarrow F$

Source	Destination	Path	Cost
A	F	$A \rightarrow C \rightarrow E \rightarrow F$	$0+2+10+3 = 15$
A	B	$A \rightarrow B$	$0+4 = 4$
A	E	$A \rightarrow C \rightarrow E$	$0+2+10 = 12$
A	C	$A \rightarrow C$	$0+2 = 2$
A	D	$A \rightarrow C \rightarrow D$	$0+2+8 = 10$

REMEMBER IN PRAYERS